```
daffy = Object.create duck
```

Is `daffy` a duck? It was created from a prototypical duck. If the `constructor.name` is the criteria for being a duck, then `daffy` isn't a duck:

```
daffy.constructor.name
# 'Object'
```

So, with all of the different techniques for constructing and modifying objects in CoffeeScript, it quickly becomes apparent that none of the approaches you try to determine whether something is the correct type are going to work. What's the alternative then? It's called duck typing.

7.4.2. How to use duck typing

The principle behind duck typing is that if there's no reliable way in the language to be sure what interface an object implements, then you should rely on the interface itself. Put simply, *if it walks like a duck and quacks like a duck, then it's a duck*:

```
class DuckRace
  duck: (contestant) ->
    contestant.quack?.call and contestant.walk?.call
  constructor: (applicants...) ->
    @ducks = (applicant for applicant in applicants when @duck applicant)

duck =
  name: 'Daffy'
  quack: ->
  walk: ->

cow =
  name: 'Daisy'
  moo: ->
  walk: ->

race = new DuckRace duck, cow
race.ducks
# [ { name: 'Daffy', quack: [Function], walk: [Function] } ]
```

If it has a call property, then it can be invoked. See section 6.4.4.

Without types, how do you have confidence that your program works correctly? You get confidence by writing tests. Good tests for the